Customer & Admin Authentication Blueprint

Dual-Token Architecture (`atoken` vs `btoken`), OTP verification lifecycle, RSA credential encryption, and silent refresh loops across `adminConsole_repo` and `iAM_repo` .

SECTION 1

Enterprise Dual-Token Architecture

Separation of Concerns

In the digital banking platform, identity verification is isolated from business domain logic. Authentication is executed against the dedicated **Identity and Access Management (IAM)** service, while all banking operations are handled through core services guarded by session tokens.

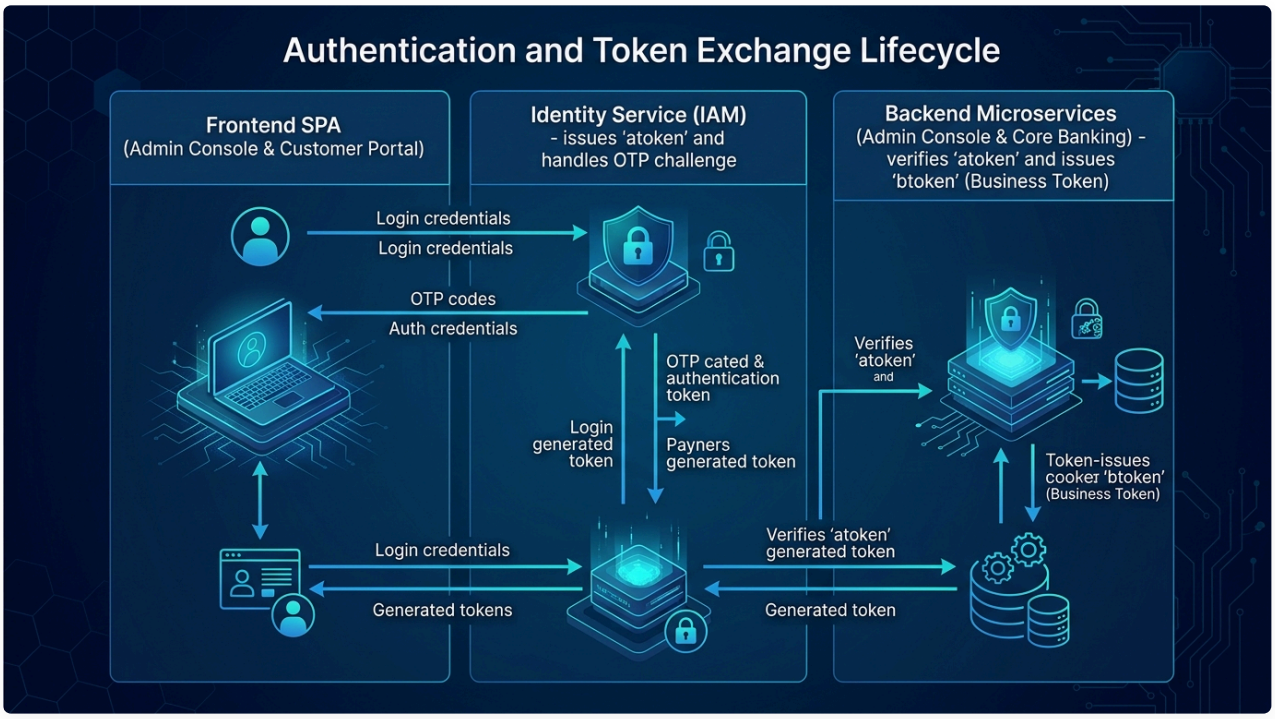


Figure 1: Architectural boundary separating IAM Identity Provider, Client SPAs, and Administrative Backend Microservices.

SECTION 2

Core Concept: atoken vs. btoken

Essential Intern Knowledge

Every intern must clearly distinguish between the identity verification token (`atoken`) and the operational session token (`btoken`):

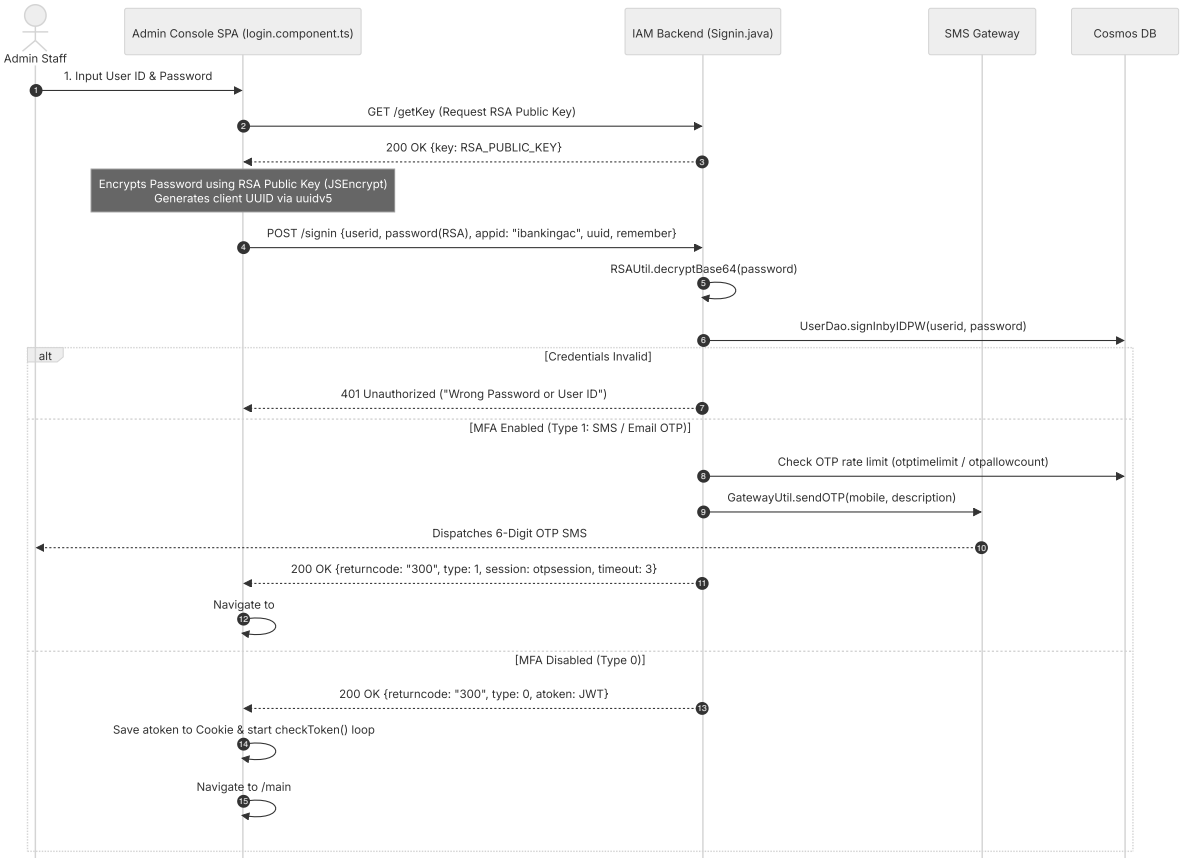
Dimension	atoken (Authentication Token)	btoken (Banking Session Token)
Issuing Authority	**IAM Backend** (`iAM_repo/ibanking_iam`) via `/signin` or `/verifyuser`	**Admin Console Backend** (`adminConsole_repo/admin_console`) via `/getmoduletoken`
Storage Location	Browser Cookie: `cookieService.set('atoken', ...)`	Web Local Storage: `localStorage.setItem('btoken', ...)`
Lifespan & Expiry	• Standard: 1 Hour • Remember Me: 30 Days • Cookie Wrapper: 365 Days • Sliding Renewal: 5-minute background check loop	• Default: 8 Hours 20 Minutes (500 minutes) • Session Inactivity: Auto-revoked upon idle timeout • Sliding Renewal: Refreshed 10 seconds before expiry on active mouse movement
Operational Scope	**Identity Proof:** Confirms user passed credential and MFA challenge. Has zero authority to trigger fund transfers or administrative mutations.	**Action Authorization:** Required by backend `AuthMiddleware` for all administrative functions. Carries CBS API access and branch permission scopes.
Payload Claims	`sub (userid), iss, iat, exp`	`sub, phone, apiToken (CBS), sid (Session ID), branchCode, ctype`

FLOW 1

Client-Side RSA

Primary Authentication & RSA Credential Handshake

Passwords are never transmitted across the network in plaintext. Before submission, the frontend queries IAM for its RSA Public Key and encrypts the password client-side:



Code Paths:

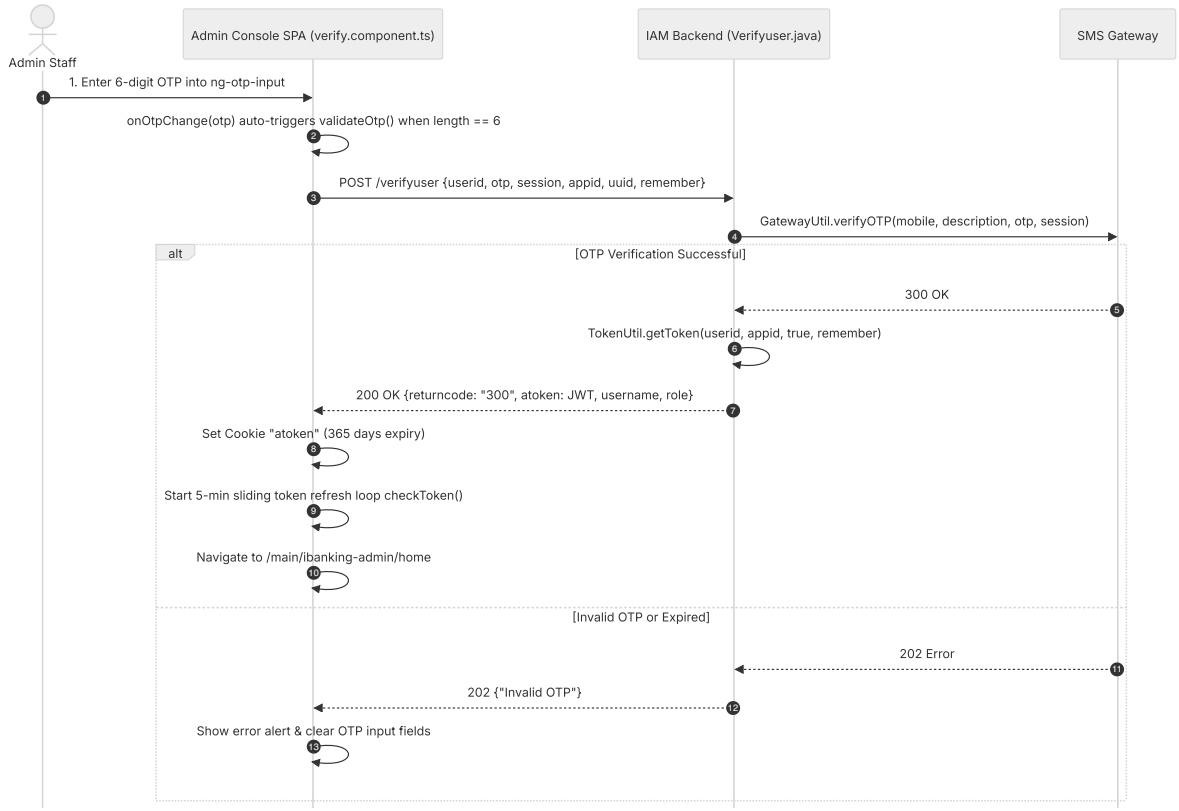
- Frontend Logic: `login.component.ts:L64-114` (`loginUser()` and `goSignIn()`)
- Backend Function: `Signin.java:L262-308` (`goSignin()` and `validateUser()`)

FLOW 2

Two-Factor Authentication

OTP Challenge & atoken Generation

Upon entering 6 digits into the segmented input box, the Angular component auto-submits the code along with the server-generated OTP session identifier:



Code Paths:

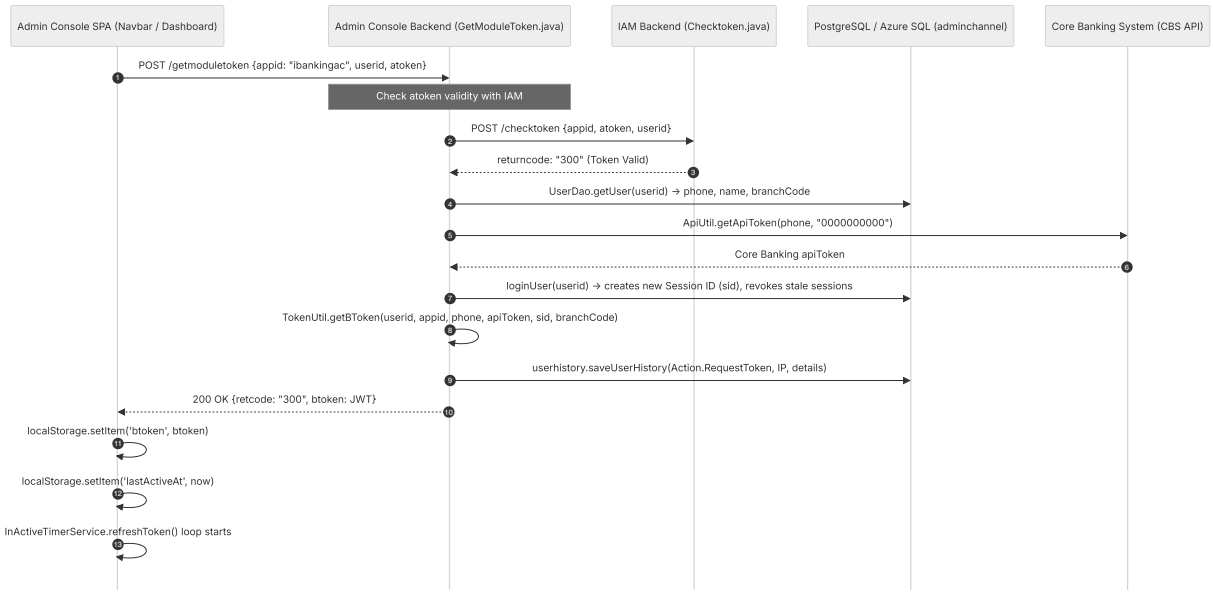
- Frontend Logic: `verify.component.ts:L125-165` (`validateOtp()`)
- Backend Function: `Verifyuser.java:L67-122`
- JWT Construction: `TokenUtil.java:L241-264` (`doGenerateToken()`)

FLOW 3

Session Registration

Token Exchange: atoken to btoken via getmoduletoken

The SPA immediately passes the `atoken` to the Admin Console backend. The backend cross-verifies the identity with IAM, connects to the Core Banking System (CBS) to retrieve a banking API token, and generates `btoken`:



Code Paths:

- Frontend Dispatcher: `navbar.component.ts:L153-180` (`getModuleToken()`)
- Backend Handler: `GetModuleTokenLogic.java:L83-145`
- Database Session Tracking: `adminchannel.loginuser` and `adminchannel.userhistory` (`Action.RequestToken`)

FLOW 4 & 5

Bearer Interceptor & Silent Sliding Refresh

Background Rotation

1. Outbound API Authorization Interceptor

In `rest.service.ts`, every outgoing HTTP call automatically appends the Bearer token:

```
getHeaders() {
  return {
    'Content-Type': 'application/json',
    'Authorization': `Bearer ${localSto
  };
}
```

2. Mouse Movement Inactivity Detection

In `inactivetimer.service.ts`, active mouse moves prevent timeouts. The token is quietly refreshed before expiration:

```
addEventListener('mousemove', () => {
  localStorage.setItem('lastActiveAt',
});
// Triggers GET /refreshtoken if user w
```

SECTION 7

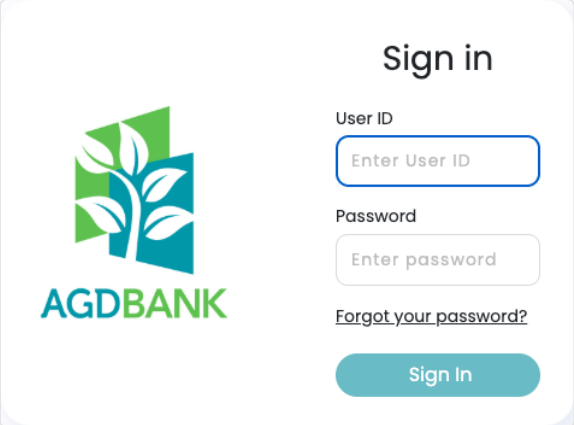
UI/UX Screen Behaviors & States

Frontend States

Screen A: Admin Sign-In (`#/iam/login`)

Actual SPA

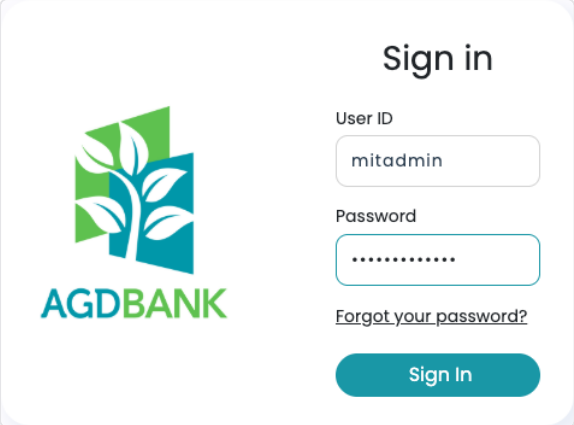
Authentic AGD Bank login card showing dynamic logo from config, User ID autofocus, and client RSA key preparation.



Screen B: Active Input & Validation

Actual SPA

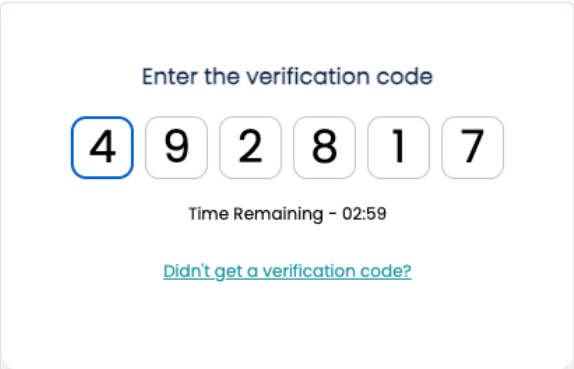
Password is typed and masked; the Sign In button activates upon validating that required fields are non-empty.



Screen C: 2FA OTP Challenge (`#/iam/verify`)

Actual SPA

Authentic 6-digit `ng-otp-input` component with countdown timer, auto-submission, and resend trigger.



SECTION 8

Hands-on Verification Checklist (Day 2 Milestone)

Required Tasks

- [ ] **Trace Frontend Handshake:** Inspect `login.component.ts` lines 64-114 and verify how `getKey` encrypts passwords with RSA before network transmission.
- [ ] **Verify Browser Storage:** Open DevTools (F12) > Application. Identify `atoken` in Cookies and `btoken` in LocalStorage.
- [ ] **Examine OTP Verification:** Review `Verifyuser.java` lines 67-122 to understand how the SMS gateway verifies OTPs and creates signed JWT claims.
- [ ] **Test Inactivity Rotation:** Review `InActiveTimerService.ts` to see how mouse movement updates `lastActiveAt` and rotates tokens silently.